

# IRE Assignment 2 Design Note

## Learning from Click-Logs on EB-NeRD and MIND

Iman Kalyan Chakraborty - 2026801003

September 14, 2026

<https://github.com/astrion-coder/cs4406m26-assignment1c1/>

## 1 Introduction

[TODO: What A2 adds on top of A1’s retrieval pipeline: behavioural features from the click-logs, a trained two-stage re-ranker, a reproduced NRMS baseline with a paired-bootstrap ablation, and a serving/scale analysis. Both datasets, large tracks, both Codabench leaderboards.]

## 2 Design

### 2.1 Click-History and Session Features (Q1)

[TODO: The fifteen features and where each comes from; recency weighting as real elapsed time on EB-NeRD versus an ordinal proxy on MIND; the five columns MIND never had; the behaviour-window boundary and why the current impression’s own read time and scroll depth are deliberately excluded.]

### 2.2 The Re-Ranker (Q2)

[TODO: Option A (GBDT/LightGBM) over Option B (small neural ranker), and why: NRMS is Q3’s baseline, so choosing the neural ranker here would have collapsed the improvement and the thing it is measured against into one model. Training hosted on Kaggle, inference local. The learning curve that settled the 400k training cap.]

#### Candidate set: why re-ranking operates on the in-view set

So Stage 1 genuinely feeds Stage 2 — just through “how well does BM25 score this article for this user” rather than “is this article in the user’s global top-200”. That is the same `score_inview` adapter A1 built, reused unchanged, which is why the two-stage framing still holds.

[TODO: Build the argument that leads into the paragraph above: the top-200 is scored against the whole catalogue (125,541 / 104,151 articles) while re-ranking must operate on `article_ids_inview` (~12 EB-NeRD, ~37 MIND); recall@200 of 0.2–3% caps any metric the second stage could produce; both Codabench leaderboards require a permutation of the in-view set, so a ranking of our own top-200 is not submittable at all; and the two membership features that do encode top-200 overlap carry exactly zero split gain on both datasets, at 2.4–4.3× chance overlap. Note this fork was identified and documented during A1 (SPEC.md Q4 §1), not retrofitted for A2.]

## 2.3 Baseline Reproduced, Then Beaten (Q3)

The baseline is `NRMSDocVec` from `ebanalyse/ebnerd-benchmark`, imported from a clone pinned at one commit and run unmodified — the notebook asserts the checked-out revision and a clean working tree, so “reproduction” means their code rather than a reimplementaion of their paper. It runs on both datasets through the same code path, because A1’s unified schema and A1 Q3’s document vectors are exactly the input `NRMSDocVec` expects; the alternative of pairing it with `recommenders-team`’s NRMS-on-MIND would have meant two implementations, a GloVe-tokenised title pipeline for MIND alone, and MIND numbers not comparable to EB-NeRD’s.

The improvement is one term. `AttLayer2` pools a user’s history order-blind and time-blind; the variant adds a recency log-weight to the attention exponent, so recency multiplies the learned attention rather than replacing it, with real elapsed time on EB-NeRD and the ordinal proxy on MIND — A2 Q1’s own two bases, not a third notion of recency. It adds no parameters, so all three variants have identical capacity, and it keeps the user vector candidate-independent, which is what makes the scoring shortcut in Section ?? possible.

The ablation has three arms rather than two, because the recency layer also masks the zero-padded history slots the baseline attends over. That is a real fix and not the recency signal, so `masked_uniform` — the same architecture fed all-zero log-weights — sits between them, and the two paired intervals separate the mask from the signal. The three point estimates decompose exactly, which the notebook asserts.

The result splits by dataset, and the three-arm ablation is what makes that readable. Every variant carries an identical 1,304,720 parameters, so none of what follows is capacity.

| dataset                   | split | baseline | masked_uniform | recency       |
|---------------------------|-------|----------|----------------|---------------|
| <code>ebnerd_large</code> | val   | 0.5701   | 0.5773         | 0.5757        |
| <code>ebnerd_large</code> | test  | 0.5835   | 0.5836         | 0.5845        |
| <code>mind_large</code>   | val   | 0.6316   | 0.6323         | <b>0.6396</b> |
| <code>mind_large</code>   | test  | 0.6307   | 0.6261         | <b>0.6349</b> |

Table 1: Per-impression AUC, 200,000 impressions per split, identical impressions across variants.

On MIND the improvement is confirmed: all four metrics on both splits give a paired 95% interval excluding zero and positive,  $\Delta\text{AUC} +0.0080$  [ $+0.0071$ ,  $+0.0090$ ] on val and  $+0.0042$  [ $+0.0033$ ,  $+0.0051$ ] on test. On EB-NeRD it is not: val agrees ( $+0.0056$  [ $+0.0046$ ,  $+0.0066$ ]) but test does not, with AUC and nDCG@5 indistinguishable from zero and MRR and nDCG@10 significantly negative. Reported as such — the assignment’s bar is met for MIND on both splits and for EB-NeRD on val only.

What the middle variant buys is the reason why. Isolating recency from the padding mask it also introduces, the two datasets attribute the same headline change to opposite causes: on MIND masking does nothing (val) or hurts (test) and the ordinal recency prior is the whole effect ( $+0.0073$  and  $+0.0088$ ); on EB-NeRD the val gain *is* the mask ( $+0.0072$ ) and the elapsed-time recency prior removes 0.0016 of it. A two-arm experiment would have reported EB-NeRD’s val result as a confirmed recency improvement, and been wrong about the mechanism.

[TODO: Two things still to write here: where these numbers sit against A2 Q2’s Stage-1 baselines and re-ranker (they were measured on the same 200,000 impressions, so the tables are directly comparable), and the early-stopping artifact from SPEC.md A2 Q3 §9 — EB-NeRD’s 07:00 split cutoff leaves the authors’ last-calendar-day rule only 3.8% of the training sample to

monitor on, against MIND’s 25.4%, which is a live alternative explanation for the EB-NeRD val/test disagreement and should be stated as one. Figures: `nrms_ablation.png`, `nrms_paired_ci.png`.]

## 2.4 Serving and Scale Analysis (Q4)

[TODO: Write out from SPEC.md A2 Q4. Served two-stage path (BM25 + embedding over the in-view set, LightGBM re-rank), 1,000 sampled requests after 50 warm-up: p99 **6.09 ms** on `ebnerd_large`, **10.43 ms** on `mind_large`, against a 100 ms SLA (16× / 9.6× headroom); 230 / 161 QPS per process; \$0.000051 / \$0.000073 per 1,000 queries at \$0.0425/vCPU-h (stated assumption). Resident serving set 0.80 / 0.68 GB, of which the embedding matrix and its unit copy are ~90%; BM25 index 84 / 85 MB; booster 374 / 320 KB.

Three findings to carry: (1) `bm25_inview` is 83% / 88% of the served path because the inverted index scores the whole catalogue to read out a dozen documents — the index choice that made A1’s candidate generation fast is the one thing holding the served path above a millisecond; a forward index would fix it. (2) The offline feature table already exceeds this machine at 1× on `ebnerd_large` (27.9 GB vs 15.7 GB) and at 2× on MIND — the measured form of every memory incident in the project, and why Q1/Q2 stream it. State clearly that this is the offline artifact; a live system derives features from history + article metadata, so the timed `feature_assembly` (a slice) is a *lower bound*. (3) Calling A1’s offline `batched_top_k` per request re-normalizes the corpus each call: 40× slower than the serving-shaped matvec, 311 / 274 ms p99, which would have shown an SLA breach at 1× — measured deliberately as the cost of reusing an offline function for serving.

Order of breakage at 10×: feature table (279 / 101 GB) first and already; brute-force embedding retrieval second (serving set 8.0 / 6.8 GB, top-200 p99 81 / 69 ms — an ANN index becomes a requirement); BM25 corpus-wide top-K third and it breaches on MIND (146 ms p99; 37 postings/doc vs EB-NeRD’s 22 — WAND/BMW or sharding). The re-ranker does not scale with anything and is the cheapest stage in the system. Figure: `serving_benchmark.png` (three panels: served-path breakdown, candidate-generation p99 against the SLA, resident memory vs this machine).]

## 2.5 Code Optimizations

[TODO: The measured memory work: the Arrow-to-Python materialization class of bug found six times over, peak stacking, the streaming join that replaced two eager collects, and BM25 determinism via `sorted(set(...))`. List format is appropriate here - this section is a changelog.]

[TODO: Q3’s scoring shortcut belongs in this changelog too. The `ebnerd-benchmark` scoring path flattens every (impression, candidate) pair into its own row, re-encoding the user’s whole 20-click history once per candidate; since `NRMSDocVec` scores by a dot product of a history-only user vector and an article-only news vector, the news encoder can run once over the whole catalogue and the user encoder once per impression, removing a factor of ~11.9 (EB-NeRD) to ~37 (MIND) of encoder work per split. The notebook checks the shortcut against `model.scorer.predict` directly — `sigmoid(dot)` within 1e-4, ranking flips counted — and reports the measured speedup. Note also that this is why the improvement was chosen to keep the user vector candidate-independent: a candidate-aware encoder would give the factor back, offline and at serving time.]

# 3 Discussion

## 3.1 Results

[TODO: Consolidated table: AUC, MRR, nDCG@5, nDCG@10 before and after re-ranking, both datasets, val and test, with bootstrap CIs and the paired CI on each difference. Plus the sample-representativeness check against A1’s full-population baselines.]

### 3.2 Why the Gain Differs Between the Two Datasets

[TODO: The substantive finding: EB-NeRD gains  $\sim 0.10$  AUC against MIND’s 0.006–0.016, because seven of fifteen features carry zero split gain on MIND versus two on EB-NeRD - and the single most informative EB-NeRD feature, `freshness_hours` at 48% of total gain, is one of the five MIND never had. Same model, same features, same hyperparameters; the ceiling is set by what the log records.]

### 3.3 Ablation and Statistical Significance

[TODO: Write out from SPEC.md A2 Q5. All seven metrics for all three methods on the identical 200k-impression sample (test): the re-ranker’s ILD is lowest on both datasets (`ebnerd_large` 0.781 vs 0.798/0.791; `mind_large` 0.785 vs 0.854/0.838) and top-10 coverage equal-or-lower, but **novelty goes opposite ways**: lower on MIND (20.10 vs 20.38/20.62, popularity-dominated, 53% of gain) and *higher* on EB-NeRD (22.33 vs 21.96/22.14, freshness-dominated, 48% of gain) — novelty is  $-\log_2$  of *train* popularity, and fresh articles have no train clicks. Same asymmetry as Q2’s accuracy gains, now on the beyond-accuracy side. State plainly that “the re-ranker reduces novelty” would be wrong on one dataset.

Slices: head is tiny on a temporal split (1.5% of EB-NeRD test; “head” is defined on train clicks and news decays), so tail  $\approx$  overall. The one negative result to report, not bury: on EB-NeRD test/head the re-ranker is **below chance** (AUC 0.416 [0.404, 0.427] vs BM25 0.596,  $n=3,032$ ). Mechanism measured: clicked head articles are a median 239 h old vs 3.1 h for tail ( $77\times$ ); the freshest in-view candidate is 1 h; the re-ranker ranks the clicked article a median 7th of 11; 87% of the time it is older than the median candidate. A freshness prior buries the evergreen story the user came back for. On val (one day after train, head articles still fresh) the re-ranker leads on head, which separates “wrong about head” from “wrong about *old*”. Cold-start exists only on MIND (6,016 test impressions): both retrieval baselines are exactly 0.5000 (empty query, full tie), the re-ranker 0.511 from popularity + position alone — not solved by anyone.

Representativeness: sampled ILD/novelty for both baselines lie inside the sample CI of A1’s full-population values on every split (extends Q2 §7’s argument to the beyond-accuracy metrics). Then: the Q3 ablation with its paired CI, and the Q9 with/without-serving-time-features run.]

### 3.4 Where This Breaks at 10x

[TODO: Feature-store joins at scale, brute-force cosine similarity needing a real ANN index, ranker-inference batching. Extend A1’s genuine memory-exhaustion incident with what the re-ranking and serving stages specifically add.]

## 4 Leaderboard Submissions

Both blind populations were scored with the full two-stage pipeline on the same machine as every other measurement in this note: Stage-1 BM25 and embedding scores over each impression’s in-view list, the Q1 behavioural features, and the Q2 LightGBM re-ranker, written in the formats A1’s submissions established. Two facts about the blind sets shaped the implementation. They carry no

click labels, so the popularity basis is the training dataset’s lookup re-keyed into the population’s article namespace (MINDlarge\_test’s ids differ from `mind_large`’s; EB-NeRD’s are shared), and `clicks_earlier_in_session` — a feature a live system would have — is imputed with its modal training value, which is prediction-identical to leaving it missing for a feature LightGBM never saw missing. And the EB-NeRD test set’s 200,000 beyond-accuracy rows all share the sentinel `impression_id` 0, so a session-feature join keyed on it alone became a  $200k \times 200k$  self-product that committed 62.9 GB and took the desktop down; every join is on `(impression_id, user_id)`.

The two scales are 205.9 M and 93.1 M candidate rows. Rather than join the feature table to the Stage-1 scores, both notebooks produce their rows in one deterministic `(user_id, impression_id)` order and align 200,000-impression chunks by position, asserting per chunk that both sides list the same impressions and candidates. Producing that order is itself the optimization worth recording: a plain polars sort of the 13.5 M-row EB-NeRD behaviors peaked at 10.1 GB on this 15.7 GB machine, so it is an external range sort (16 key-range buckets, row-group-batched reads, one writer per bucket) at a 2.55 GB peak, row-identical to the whole-file sort. The user-contiguous order is also what lets the BM25 adapter’s one-entry cache score each user’s corpus once rather than per impression: Stage-1 plus re-ranking ran at 4,356 impressions/s on EB-NeRD (51.8 min for 13,536,710 impressions) against the  $\sim 430/s$  of the non-contiguous evaluation sample in Q2. Feature building took 230 min (980/s) and 15 min (2,664/s); the EB-NeRD build held 10.4–12.8 GB resident, the same envelope as the `ebnerd_large` build in Q1.

Each zip was checked line by line against `behaviors.parquet` (one line per row in file order, matching id and candidate count, a valid permutation) and, independently of the notebook, against A1’s embedding-baseline file for the same population: the re-ranker picks the same top candidate as the embedding baseline for 43.6% of MIND impressions and 24.2% of EB-NeRD’s, with 8.0% and 0.5% identical permutations — a different ranking of the same candidate lists, in proportion to how much of each booster’s gain the embedding score carries (12.5% and 0.1%).

[TODO: Screenshots from both Codabench competitions, with the two-stage pipeline’s score next to A1’s BM25/embedding entries.]